



Codificación agéntica

¿el *Silver Bullet*?

En 1986, Fred Brooks sentenció que el software no tendría bala de plata: ninguna tecnología entregaría un orden de magnitud de mejora en productividad, confiabilidad ni simplicidad.

Esencia y accidente

Esencia

Las dificultades **inherentes** a la naturaleza del **software**. Brooks las agrupa en cuatro:

Complejidad

Conformidad

Cambiabilidad

Invisibilidad

Ninguna desaparece por progreso técnico.

Accidente

Las dificultades **contingentes** — presentes hoy, pero no inherentes.

Lenguajes de bajo nivel, herramientas primitivas, traducción a interfaces arbitrarias, ciclos de compilación.

Estas sí se pueden solucionar. Y se han solucionado.



La IA solucionó lo accidental

Qué facilita

- Traducción entre lenguajes y stacks
- Plantillas, boilerplate, plumbing
- Adaptadores a APIs e interfaces arbitrarias
- Migraciones y transformaciones de datos
- Búsqueda y refactor a escala del repo
- Generación de tests y documentación
- Explicación y síntesis de código ajeno

Impacto

Brooks (1986) advertía que las ganancias en lo accidental ya estaban agotándose.

Cuarenta años después, la IA abrió **otro orden de magnitud** justo ahí.

Amodei · 90 % del código escrito por IA en meses
(CFR, 2025)

Cherny · 100 % de mi código lo escribe Claude Code
(Lenny's, 2026)

DORA 2024 · aumento medible de throughput con GenAI



La esencia sigue intacta

Complejidad

La IA traslada la complejidad; no la reduce. Más código generado más rápido — y los estados, acoplamientos y race conditions siguen ahí.

METR 2025: desarrolladores experimentados con IA en código maduro fueron **19 % más lentos**.

Conformidad

La IA traduce arbitrariedad a código en minutos. Pero la arbitrariedad — regulación, protocolo legacy, API propietaria — sigue siendo arbitraria.

Baja el costo de conformarse, no elimina aquello a lo que hay que conformarse.

Cambiabilidad

El costo marginal de rehacer un módulo colapsa. Pero **DORA 2024** midió la *GenAI Anomaly* — más cambio, menos estabilidad.

La presión para cambiar no desaparece; se amplifica.

Invisibilidad

Doble filo. Como lente — explicar, mapear, documentar — la IA hace el software más visible.

Como generador sin lector, “*forget that the code even exists*”, lo hace menos.





Dr. Erik Meijer

Diseñador de Visual Basic, C#, Haskell, LINQ y Hack · 2024

youtube.com/watch?v=SsJqmV3Wtkg · citado en Kim & Yegge,
Vibe Coding

“Los días de escribir código a mano están llegando a su fin.”



Andrej Karpathy

Cofundador de OpenAI · 2 de febrero de 2025

x.com/karpathy/status/188619218480814938

3

“Hay una nueva forma de codificar a la que llamo *vibe coding*: te entregas por completo a las vibras, abrazas lo exponencial y olvidas que el código siquiera existe.”



Dario Amodei

CEO de Anthropic • Council on Foreign Relations • 10 de marzo de 2025

“Estamos a tres o seis meses de un mundo en el que la IA escribe el 90 % del código. Y en doce meses, podríamos estar en un mundo en el que la IA escribe prácticamente todo el código.”



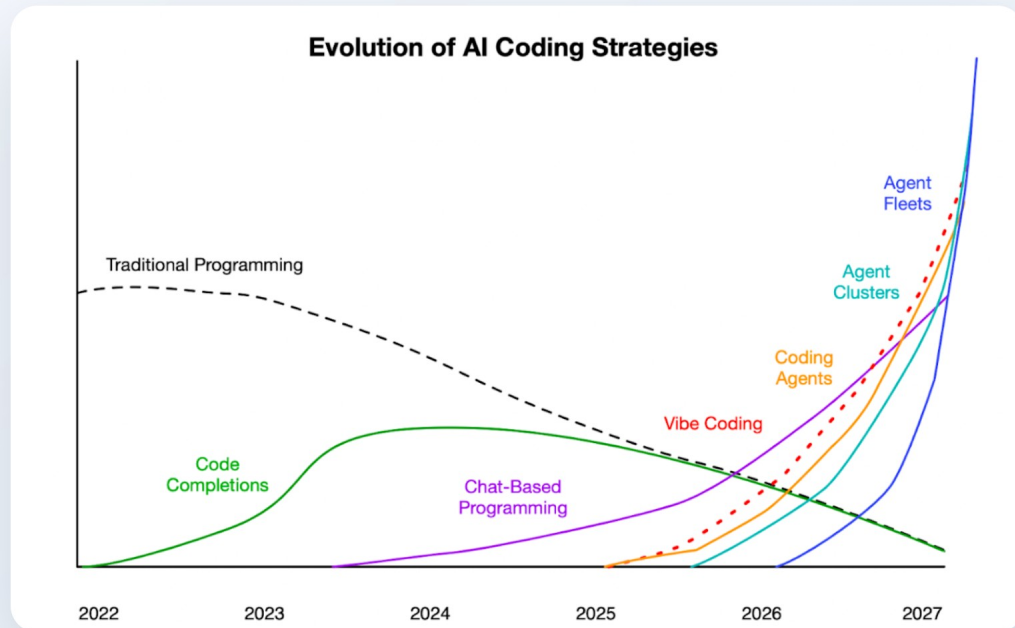
Boris Cherny

Head of Claude Code en Anthropic • Lenny's Podcast • febrero de 2026

youtu.be/We7BZVKbCVw

“El 100 % de mi código lo escribe Claude Code.
No he editado una sola línea a mano desde noviembre.
Cada día envío 10, 20, 30 pull requests.”

ACTO I · LAS VOCES DE LA PROMESA

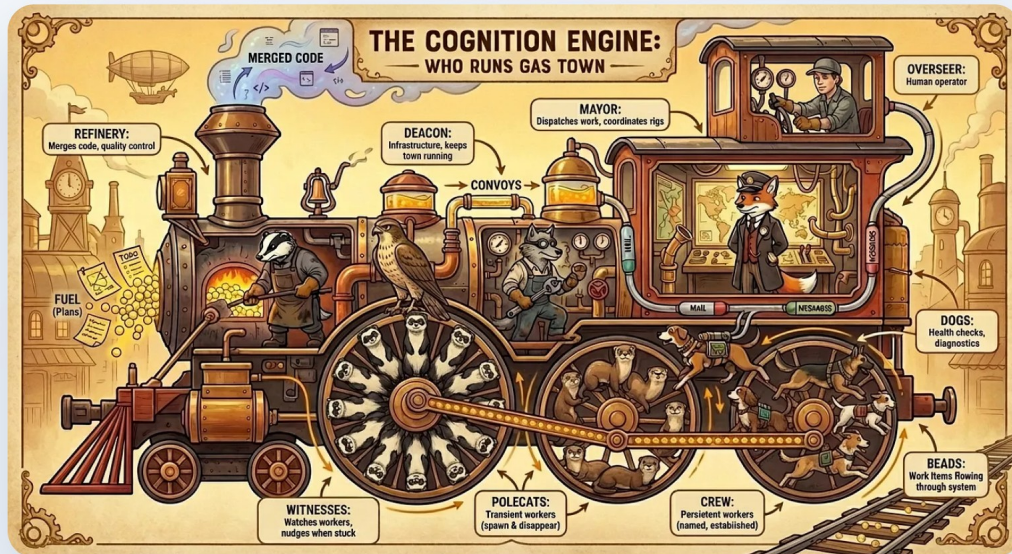


Steve Yegge

Coautor de *Vibe Coding* · “Revenge of the Junior Developer” · marzo de 2025

sourcegraph.com/blog/revenge-of-the-junior-developer

“No caigas en la tentadora trampa de aplazar el trabajo. Decir ‘en seis meses será mucho más rápido, así que voy a posponer esto seis meses’ es como decir ‘voy a esperar a que el tráfico baje’. Tu trayecto será más corto, claro. Pero llegarás de último.”



Steve Yegge

Welcome to Gas Town · enero de 2026

steve-yegge.medium.com/welcome-to-gas-town-4f25ee16dd04

“El foco es *throughput*: creación y corrección a la velocidad del pensamiento. En Gas Town dejas que Claude Code haga lo suyo. Tú eres el Product Manager y Gas Town es un compilador de ideas.”

**Pero no todo
es promesa**





Jessie Young

Principal Engineer, GitLab · citada en *Vibe Coding* (Kim & Yegge, 2025)

“Le dije a mi equipo:
no habrá *vibe coding* mientras yo esté
de guardia.”

Pertenece al grupo que recibe la alerta cuando un servicio core de GitLab se cae. Si codificar se vuelve sin fricción, **la responsabilidad se difumina**.

Su pesadilla: que cada equipo apague el cerebro, empuje código a medio hacer a producción — y a ella la despierten a las 2 a.m. a apagar incendios.

DORA 2024: la anomalía GenAI

DORA (DevOps Research and Assessment) es el programa de investigación fundado por Nicole Forsgren, Jez Humble y Gene Kim, hoy parte de Google Cloud. Publica cada año el *Accelerate State of DevOps Report* con base en decenas de miles de profesionales.

-1.5 %

throughput de entrega

-7.2 %

estabilidad de entrega



Estudio METR, julio de 2025

METR (Model Evaluation & Threat Research) es una organización sin ánimo de lucro dedicada a la evaluación empírica de sistemas de IA. Ensayo controlado aleatorizado: 16 desarrolladores experimentados, 246 tareas reales.

esperaban

+24 %

más rápidos

creyeron que fueron

+20 %

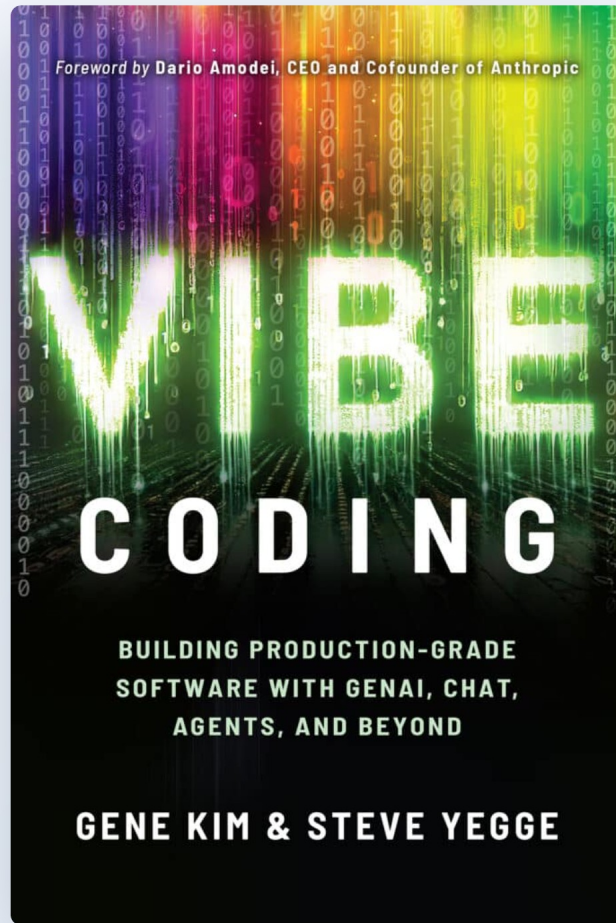
más rápidos

en realidad fueron

-19 %

más lentos





Vibe Coding

Building Production-Grade Software With GenAI, Chat, Agents, and Beyond

Gene Kim · autor de *The Phoenix Project* y *The DevOps Handbook*

Steve Yegge · Amazon, Google, Grab, Sourcegraph

IT REVOLUTION · 2025



FAAFO

F

Fast

Rápido

A

Ambitious

Ambicioso

A

Autonomous

Autónomo

F

Fun

Divertido

O

Optionality

Opcionalidad

El acrónimo con el que Kim y Yegge describen el beneficio del vibe coding bien hecho.



Tres loops de desarrollo

INNER LOOP

Segundos a minutos

La conversación momento a momento con el agente: escribir un prompt, recibir una propuesta, aceptar o corregir. Es donde la mayoría ya aprendió a trabajar con IA — y donde terminan la mayoría de los pilotos sin madurar.

MIDDLE LOOP

Horas a días

Sesiones del agente que entran al armario y olvidan todo al salir. Gestión de contexto, memoria de la sesión, planeación táctica. Aquí aparecen los primeros problemas serios: contextos que se pierden, trabajo que se repite.

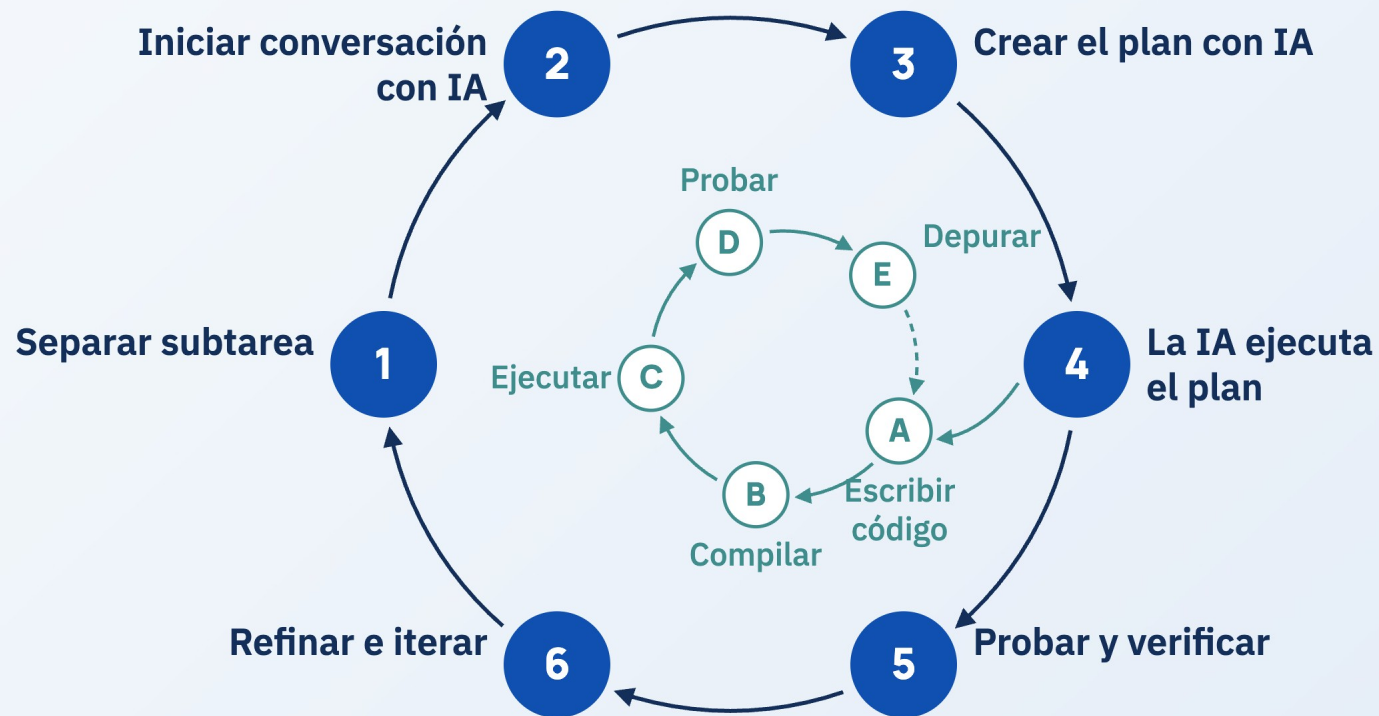
OUTER LOOP

Semanas a meses

Arquitectura, workflows, sostenibilidad del sistema. Prácticas que atraviesan varios agentes y varios desarrolladores. Aquí se decide si el agente acelera al equipo o amplifica su fragilidad.



El Vibe Coding Developer Loop



1–6 · Loop externo agéntico.

Descomponer, conversar, planear, verificar, iterar.

A–E · Inner loop tradicional, embebido en el paso 4.

Lo que siempre hicimos cabe ahora en un solo paso

Principios del outer loop • el rol

01

Asumimos el rol de orquestador

Ya no picamos verduras: diseñamos el menú, gestionamos la cocina y aseguramos que cada plato cumpla el estándar Michelin.

02

Tratamos a la IA como compañera – rápida, incansable

La guiamos, iteramos con ella, le damos contexto como a un aprendiz humano. Y le pedimos rehacer el trabajo sin dudar: no tiene sentimientos que herir.

03

Descomponemos sin piedad

Task graphs y *tracer bullets*: partimos la visión ambiciosa en trozos manejables por IA y probamos los caminos críticos temprano.



Principios del outer loop · la responsabilidad

04

Nuestra cocina, nuestras reglas, nuestra responsabilidad

Cada línea de código, cada bug, cada factura de un proceso de IA fuera de control cae sobre nuestro *toque*. Revisamos y validamos con rigor.

05

Le exigimos a la IA todo lo que necesitamos

El *sous chef* prospera con el volumen. Lo usamos para explorar opciones, refactorizar a fondo y generar los tests y la documentación que nunca tuvimos tiempo de hacer.

06

Pensamos como team lead

Al orquestar varios agentes a la vez adoptamos desarrollo en paralelo, integración de cambios y fijación de estándares. Ahí escala la ambición — y el FAAFO.



Caso Adidas · 700 desarrolladores

500 → 700

desarrolladores en un año

91 %

satisfacción

82 %

engagement diario

Happy Time: 47 % → 65 % (2018 → 2024)

El tiempo en *coding, testing, analysis, design, documentation* — la zona — subió 18 puntos.

Pero los promedios engañan: dentro del piloto convivían dos poblaciones muy distintas.



Dos arquitecturas, dos realidades

Arquitectura desacoplada

E-commerce · *fast feedback loops* · deploys frecuentes

80 % Happy Time
FAAFO funcionó

ERP legacy acoplado

Deploys dos veces al año · testing que tomaba días

30 % Happy Time
FAAFO falló

“Si les hubiera ofrecido Copilot a esos equipos, me habrían insultado. Me habrían dicho: Fernando, ¿estás loco? Arregla el entorno, arregla el testing primero.”

— Fernando Cornago, VP of Software Engineering, Adidas

El diferenciador no fue el *skill* (capa 1) ni el *tooling* (capa 2): fue la arquitectura (capa 3). **Theory of Constraints**: toda mejora que no se hace en el cuello de botella es una ilusión.



Nuevas habilidades · leer, escribir, enfocar

Las herramientas cambian rápido. Por eso los principios tradicionales de ingeniería juegan un rol al menos tan grande — si no mayor.

01

Lectura de código

Cientos de cambios, miles de líneas al día. Necesitamos leer rápido, con ojo de águila para las anomalías. *Los humanos somos el backstop cuando la IA falla.*

02

Comunicación precisa

Nuestro éxito depende de cómo dirigimos a la IA. Escribir claro es pensar claro. *“Writing is thinking.”* — David McCullough

03

Concentración

Mantener problemas cada vez más grandes — y más de ellos — en la cabeza mientras orquestamos varios agentes en paralelo.



Nuevas habilidades · integrar, diseñar, emprender

04

Control de versiones

Mergeamos cambios de múltiples fuentes humanas y de IA. *Three-way* y *N-way merges* requieren método, no reflejos.

05

Arquitectura y modularidad

Cómo se diseñan sistemas grandes, cómo se habilita independencia, cómo se comportan bajo carga. Más importante que memorizar sintaxis.

06

Conciencia de negocio

Equipos pequeños *AI-driven* pueden disrumpir mercados. Modelos de negocio, *pitch*, colaboración multidisciplinar.



Para empezar hoy

1 Empecemos pequeño, empecemos hoy

Démosle a la IA una tarea acotada. Veámosla tropezar, corriámosla, tensemos el loop. Luego dupliquemos el alcance. *“A la décima iteración, la conversación ya no se siente como usar una herramienta — se siente como dirigir.”*

2 Documentemos las reglas de la cocina

Repositorio impecable. *Pager test*: si un commit escrito por un agente puede despertar a alguien a las 2 a.m., el nombre del dueño tiene que quedar clarísimo.

3 Contagiamos la curiosidad

Demos en el almuerzo, PRs al catálogo de prompts del equipo, tableros de consumo compartidos. La forma más rápida de subir la calidad es que la exploración se contagie.

4 Guardemos el mindset, no la matriz de features

Los modelos, los contextos y las flotas de agentes van a seguir mutando. Lo que nos sostiene es la mentalidad: **modularidad, feedback, juicio.**



¿Qué hacemos para vivir del lado del Happy Time?

Event-Driven Architecture

Event Sourcing

Serverless

Infrastructure as Code



1/4 Event-Driven Architecture

El chasis del desacople

- | Cambio sin coordinar con cinco equipos.
- | Módulos que hablan por mensajes.
- | Una parte falla sin tumbar el sistema.
- | Sumar un consumidor no toca al productor.



2/4 Event Sourcing

La historia es la verdad

| Se guarda lo que pasó.

| El esquema evoluciona sin tumbar nada.

| El agente reconstruye el porqué.

| La auditoría es el sustrato.



3/4 Serverless

El código es un pasivo

Cada línea que se escribe se mantiene, se defiende y se paga.

El cloud opera lo que no queremos operar: servidores, parches, capacidad, escalado.

Se paga por uso real — no por capacidad reservada.

Menos código propio: menos bugs, menos deuda, menos riesgo.

“Un enfoque serverless-first asegura carga operativa baja y ciclos de retroalimentación rápidos.” — Anderson, McCann, O'Reilly · *The Value Flywheel Effect*, 2022



4/4 Infrastructure as Code

Clickops no escala, no se automatiza

- | La infraestructura se declara en código, se revisa en PR, se despliega por pipeline.
- | Clickops son pasos manuales que ningún pipeline puede reproducir.
- | El agente lee y modifica configuración declarativa; el portal gráfico lo deja ciego.



DORA como espejo

Si es Happy Time, se ve en las métricas

Lead time

Qué tan rápido un cambio llega a producción

Deploy frequency

Cuántas veces por día se despliega

MTTR

Cuánto se tarda en recuperarse cuando algo falla

Change failure rate

Qué porcentaje de cambios rompe producción



El nuevo junior

**La IA es el desarrollador junior que
cada uno acabamos de contratar**



Las tres capas del trabajo

1 **El trabajo mismo**
donde se crea valor

2 **Las herramientas**
con qué se hace

3 **La arquitectura de la organización**
cómo todo se conecta



Qué incluye la capa 3

Elemento	Qué define
Arquitectura de sistemas	cómo se conectan los componentes
Diseño organizacional	cómo están estructurados los equipos
Protocolos de comunicación	quién le habla a quién, sobre qué, con qué frecuencia
Flujos y procesos	cómo se mueve el trabajo de inicio a fin
Estándares e interfaces	las reglas acordadas entre áreas
Normas de liderazgo y cultura	cómo actúan y reaccionan las personas

Ley de Conway: la arquitectura del software imita la estructura de comunicación del equipo.



La tercera capa es nuestra

La IA va a cambiar nuestras decisiones de capa 3

Dirigir al propio agente se vuelve la capa 3 individual de cada dev.

Coordinar a varios devs que ya coordinan agentes es un problema abierto.

Cuando codificar deja de ser el cuello de botella, la organización lo es.



Accionables de la capa 3

Organizar agentes es organización, no tooling

Subagentes especializados	cada rol con su contexto y competencia: planner, test-writer, implementer, reviewer.
Generadores y verificadores	quien genera no verifica. TDD como sustrato.
Documentación compartida	CLAUDE.md, ADRs, bitácora. El agente lee lo que el equipo decidió.
Paralelismo bien diseñado	issues con dependencias declaradas, worktrees aislados.
Integración de verificación	CI/CD, revisión de PR, métricas DORA.



Debemos abordar proyectos ambiciosos

Los agentes nos permiten emprender trabajo que antes no era viable

Cobertura de pruebas exhaustiva en módulos que llevan años sin test.

Documentación técnica y funcional viva — que a su vez alimenta al próximo agente.

Pruebas end-to-end automatizadas donde hoy todo se valida a mano.

Desacople gradual de módulos atados a stacks legacy.

Productos nuevos autocontenidos — explorar sin pagar el costo de antes.



Nuestro compromiso

Aprender

a dirigir agentes — cada uno, en su propio contexto.

Aplicar

harness engineering — el oficio de domar la IA para que produzca trabajo confiable.

Formalizar

la capa 3 — documentación viva, ADRs y protocolos que hagan escalable el trabajo con agentes.



Fuentes · libros y ensayos

LIBROS

Gene Kim & Steve Yegge. *Vibe Coding: Building Production-Grade Software With GenAI, Chat, Agents, and Beyond*. IT Revolution, 2025.
Prólogo: Dario Amodei.

Fred Brooks. *No Silver Bullet — Essence and Accident in Software Engineering*, 1986. En la 2ª ed. de *The Mythical Man-Month*, 1995.

Gene Kim & Steven Spear. *Wiring the Winning Organization*. IT Revolution, 2023.

David Anderson, Mark McCann, Michael O'Reilly. *The Value Flywheel Effect*. IT Revolution, 2022.

BLOGS Y ENSAYOS

Andrej Karpathy, X/Twitter, 2 feb 2025 — definición original de vibe coding.

Steve Yegge, *Revenge of the Junior Developer*, Sourcegraph, mar 2025.

Steve Yegge, *Welcome to Gas Town* (ene 2026) y *The AI Vampire* (feb 2026), Medium.

IT Revolution, *The Vibe Coding Loop* y *The Three Developer Loops*, 2025.



Fuentes · voces y estudios

ENTREVISTAS Y CONFERENCIAS

Dario Amodei, Council on Foreign Relations, 10 mar 2025.

Boris Cherny, *Head of Claude Code: What happens after coding is solved*, Lenny's Podcast, feb 2026 · youtu.be/We7BZVKbCVw

Dr. Erik Meijer — charla en YouTube, 2024 · youtube.com/watch?v=SsJqmV3Wtkg

VOCES CRÍTICAS

Jessie Young et al., *No Vibe Coding While I'm On Call*, IT Revolution — Enterprise Technology Leadership Journal, Fall 2025.

ESTUDIOS

DORA / Google Cloud, *Accelerate State of DevOps 2024* — GenAI Anomaly.

DORA / Google Cloud, *Impact of Generative AI in Software Development*, 2024.

METR, *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*, jul 2025 · arxiv.org/abs/2507.09089



Gracias

Preguntas

